Hochschule für
Wirtschaft und Recht Berlin
Berlin School of Economics and Law

Embedded Systems Engineering Introduction

Prof. Dr.-Ing. Sebastian Schlesinger
Professor of Business Computer Science
Cybersecurity and Embedded Systems Engineering

Überblick über die gesamte Vorlesung

Hochschule für Wirtschaft und Recht Berlin
Berlin School of Economics and Law

Eingebettete Systeme

- Definition
- Eigenschaften
- Anforderungen

Entwurf Eingebetteter Systeme

Modellierungssprachen

- Modell-basierte Entwicklung
- Endliche Automaten
- Statecharts
- Timed Automata
- Signalfluss-orientierte Sprachen (Simulink)
- SystemC

Implementierung

- Programmier-sprachen
- Betriebssysteme
- Synthese

Validierung und Verifikation

- Testen
- Simulation
- Model Checking

Überblick über die gesamte Vorlesung

Hochschule für Wirtschaft und Recht Berlin
Berlin School of Economics and Law

Eingebettete Systeme

- Definition
- Eigenschaften
- Anforderungen

Entwurf Eingebetteter Systeme

Modellierungssprachen

Implementierung

Validierung und Verifikation

Übersicht: aktuelle Vorlesung

Hochschule für Wirtschaft und Recht Berlin
Berlin School of Economics and Law

- Organisatorisches

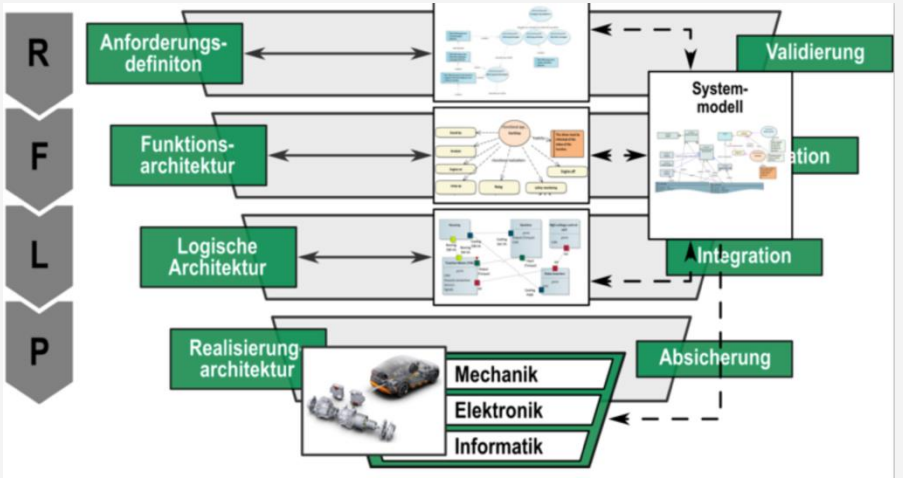
- Eingebettete Systeme
  - Was sind eingebettete Systeme?
  - Warum sind eingebettete Systeme wichtig?
  - Besonderheiten eingebetteter Software
  - Herausforderungen im Software Engineering

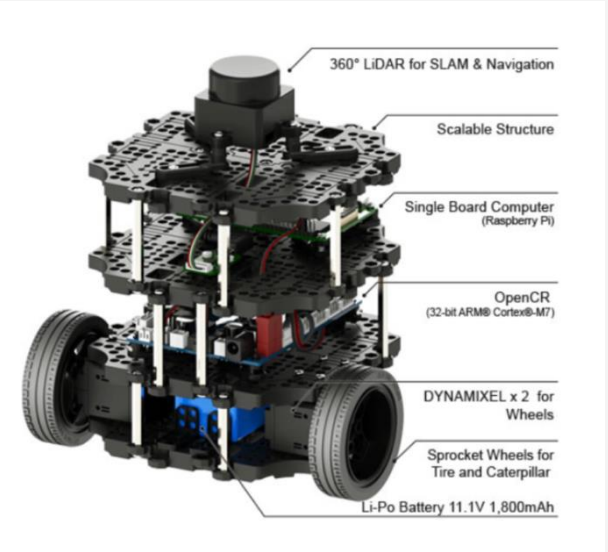
- Vorlesungsinhalte
- Literatur

Prüfungsmodalitäten

1. Teil: Theorie
- Model Driven Engineering
- Fokus: Modellierung und Verifikation von ES

2. Teil: Implementierung
- Robotikprojekt (Turtlebots, ROS2) realisieren
- Projekte wie SLAM, ggf. Kollaboration der Roboter modellieren und implementieren
- Test am Ende des Theorieteils (50%)
- Systemmodellierung und Implementierung
  - Gruppenarbeit in verteilten Rollen
  - 3 TN pro Gruppe
  - Ziel: Robotik-Praktikum





Quellen: https://www.imw.tu-clausthal.de/forschung/schwerpunkte/modellbasierte-systementwicklung, https://www.generationrobots.com/de/402707-turtlebot3-burger.html?srsltid=AfmBOopMWXNdvS3TnZ2MVhOo0FZ8WZISRl6hLkrZlIKMWRieHfgplkr3

Übersicht: aktuelle Vorlesung

Hochschule für Wirtschaft und Recht Berlin
Berlin School of Economics and Law

- Organisatorisches
- Eingebettete Systeme
  - Was sind eingebettete Systeme?
  - Warum sind eingebettete Systeme wichtig?
  - Besonderheiten eingebetteter Software
  - Herausforderungen im Software Engineering
- Vorlesungsinhalte
- Literatur

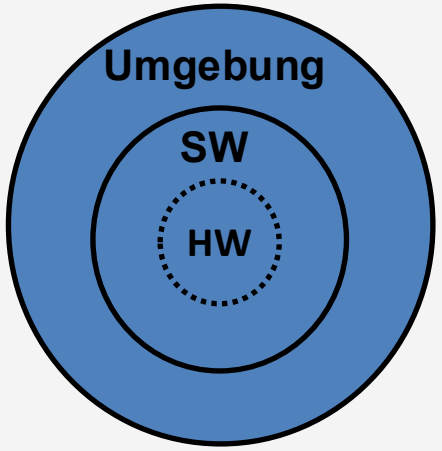
Was sind eingebettete Systeme?

Hochschule für
Wirtschaft und Recht Berlin
Berlin School of Economics and Law

Was stellen Sie sich unter eingebetteten Systemen vor?
Nennen Sie Beispiele!

Was sind eingebettete Systeme?

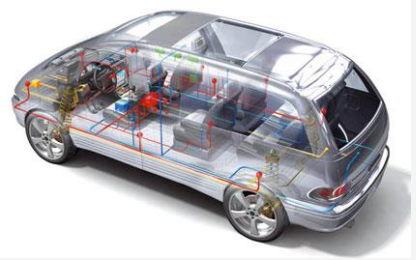
Eingebettete Systeme steuern eine (analoge) technische Umgebung mit digitalen Komponenten (HW und SW)



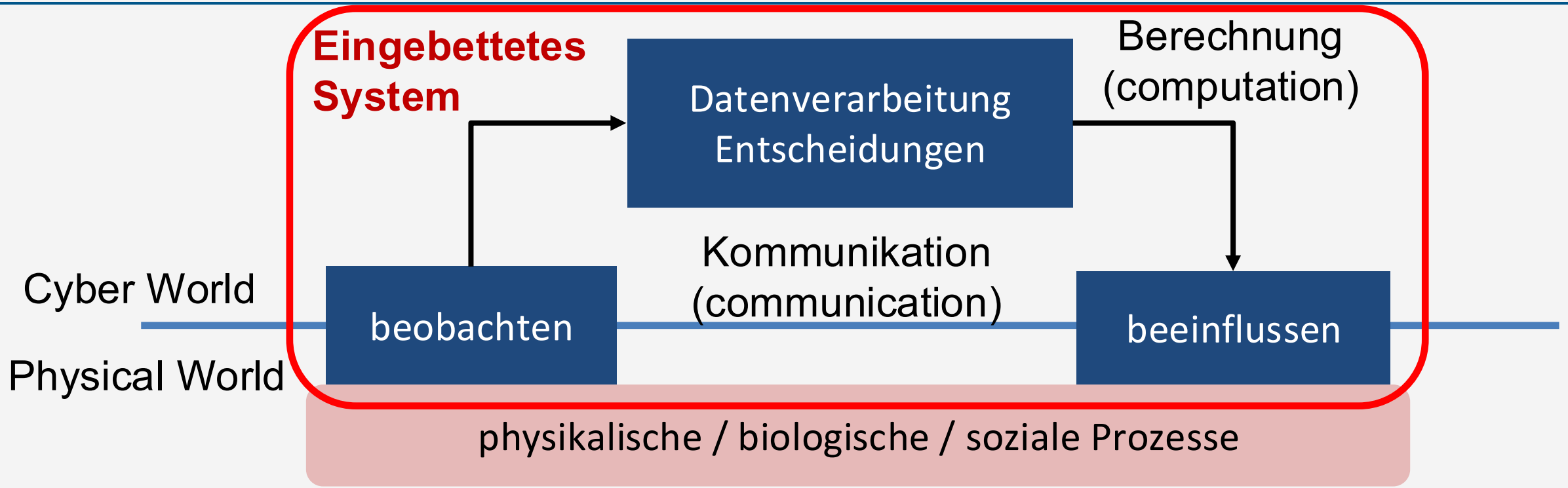
- häufig nicht sichtbar
- Interaktion mit der technischen Umgebung über Sensoren und Aktoren

Beispiele eingebetteter Systeme

- Automobiltechnik
  - Navigationssysteme, ABS, ESP, …
- Luft- und Raumfahrttechnik
- Unterhaltungselektronik
- Verkehrstechnik
- Infrastrukturen
  - Wasser, Energie, …
- Haushalt
- Medizintechnik
- Industrielle Fertigung
- Cyber-Physical Systems und Internet of Things (IoT)



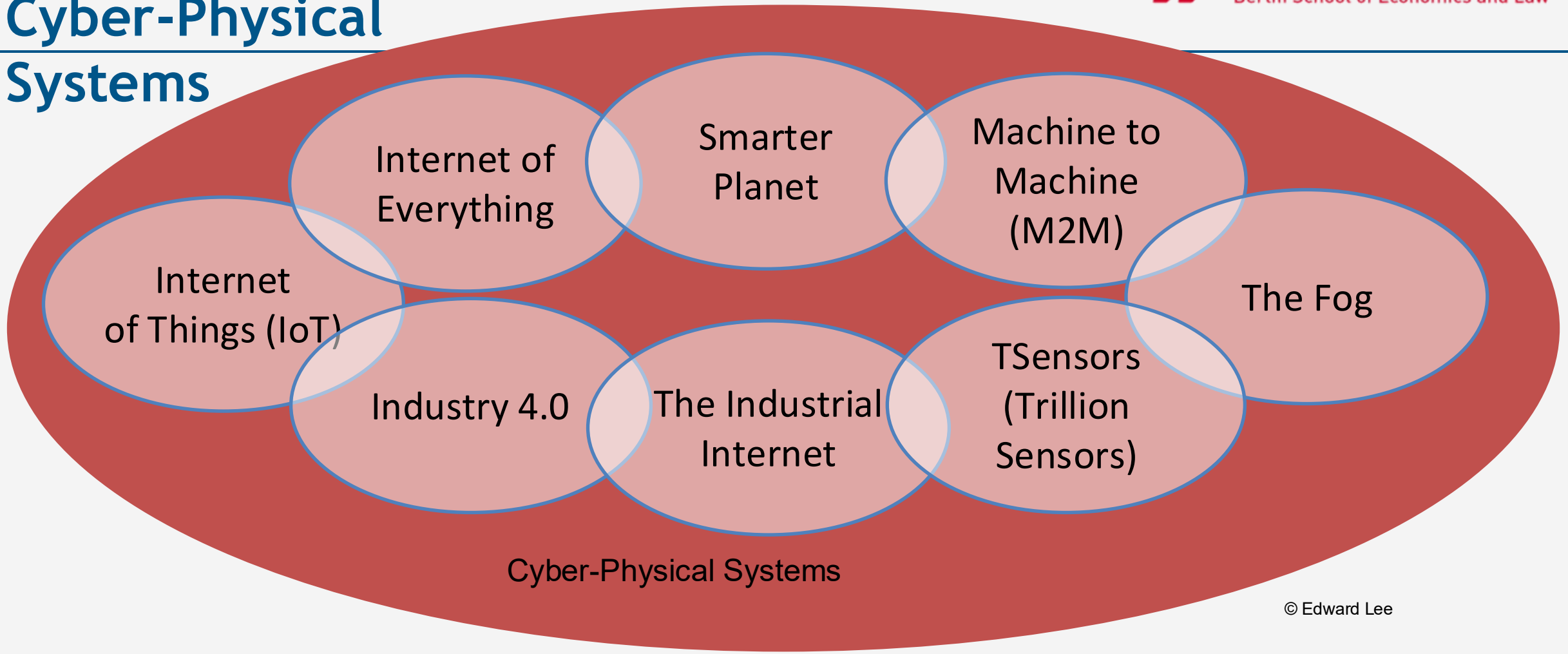
Cyber-Physical Systems



"Use Feedback to influence the dynamics of the physical world by taking smart decisions in the cyber world."

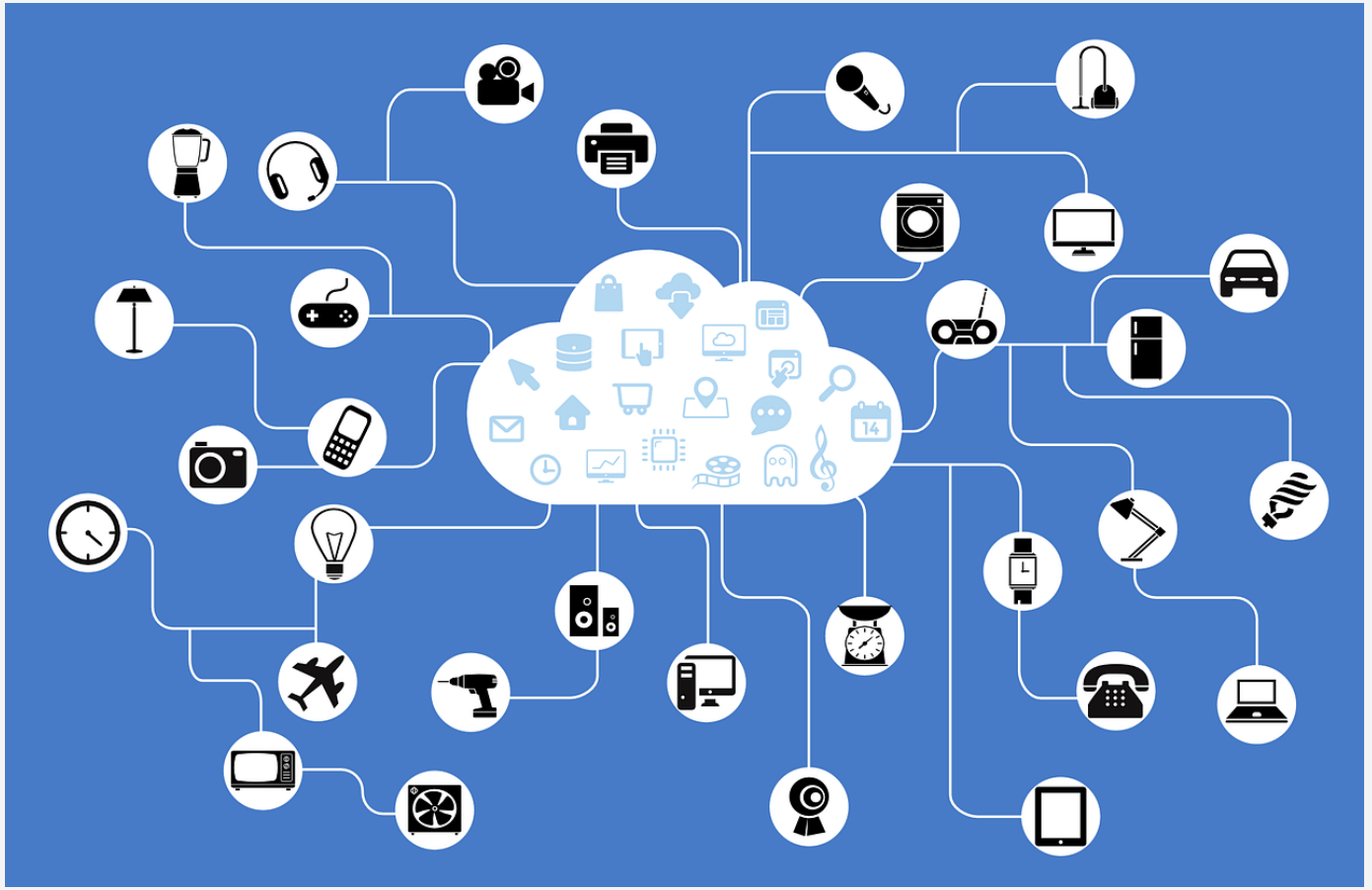
© Lothar Thiele

Cyber-Physical Systems

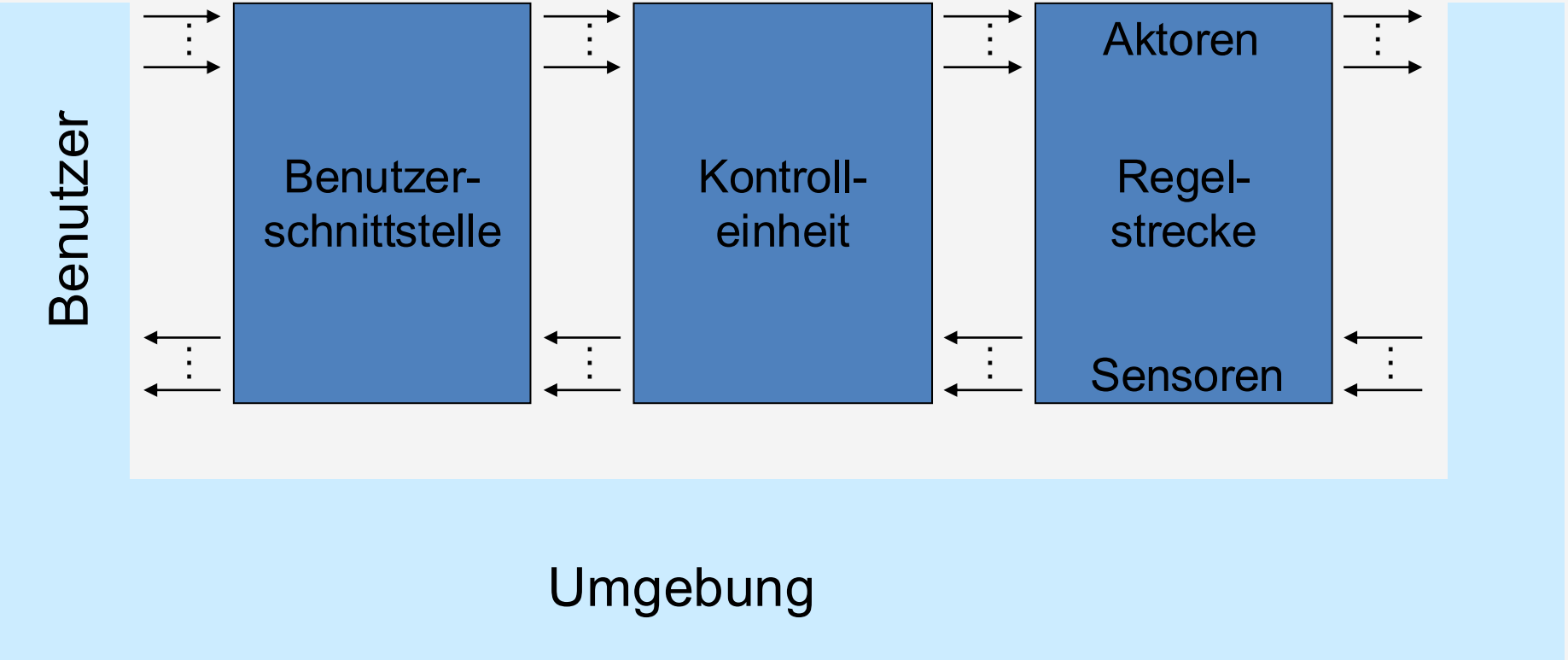


© Edward Lee

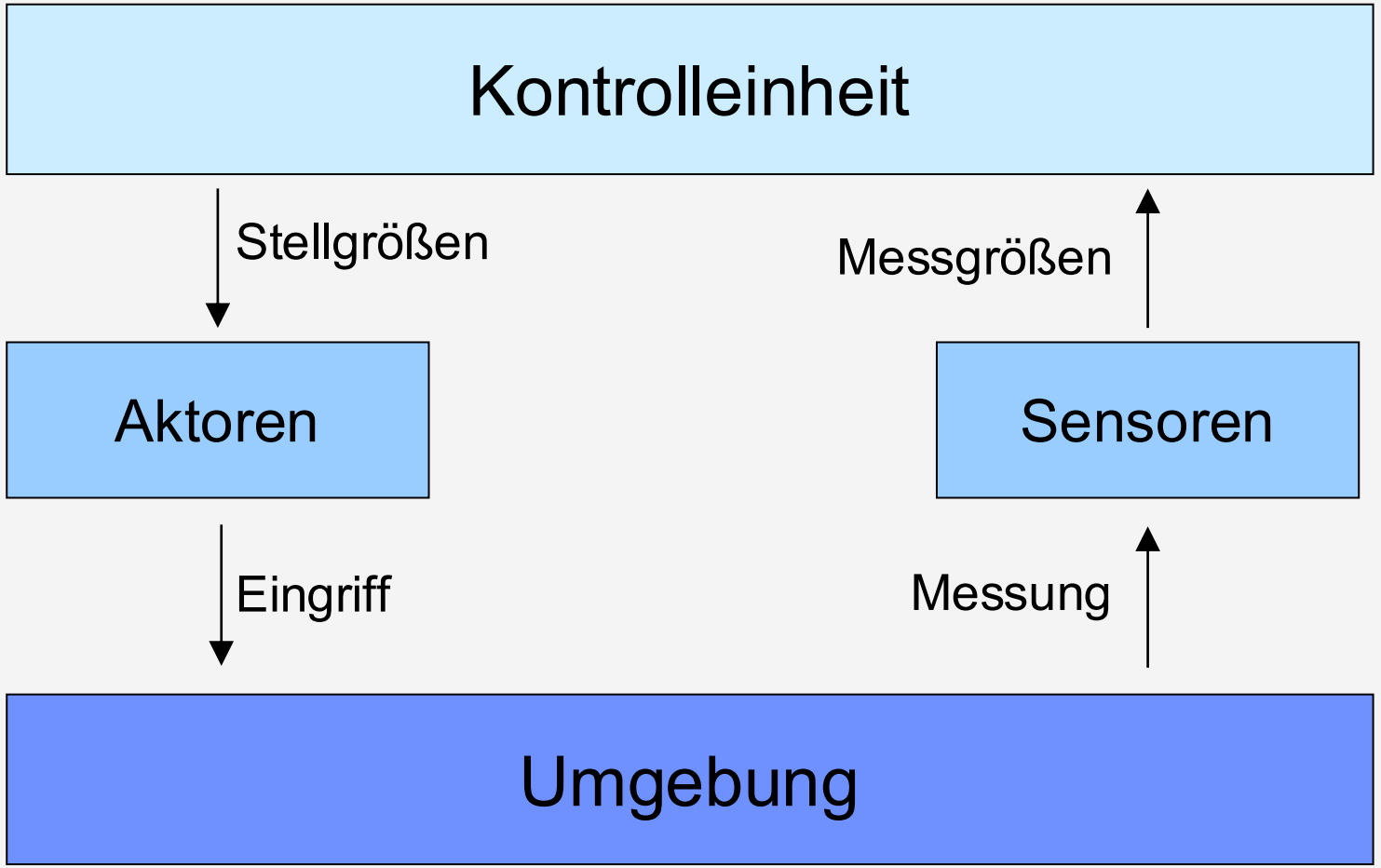
Internet of Things (IoT)



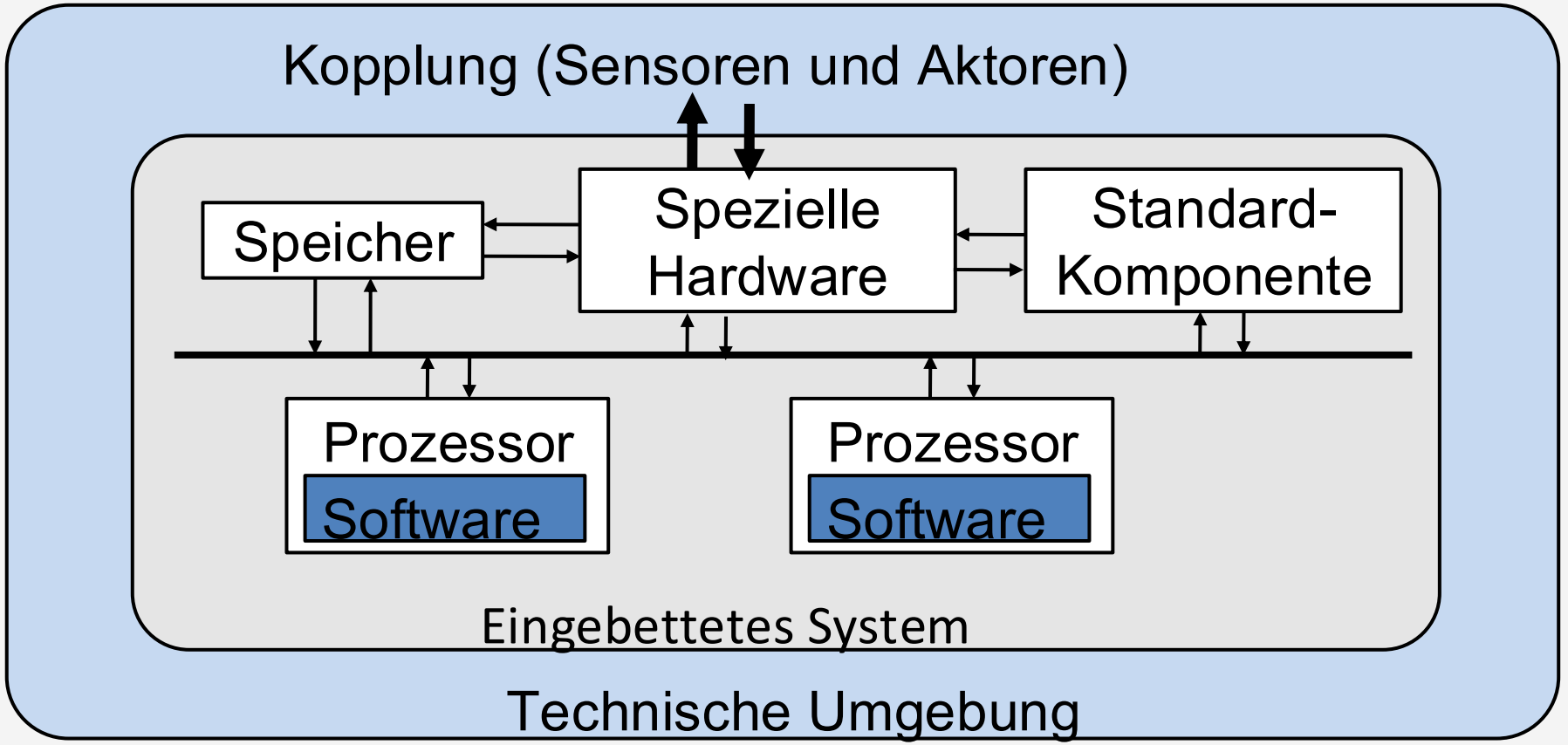
Struktur eingebetteter Systeme



Wirkungskette System - Umgebung



Architektur Eingebetteter Systeme



Diskussion

Hochschule für Wirtschaft und Recht Berlin
Berlin School of Economics and Law

Was sind ES?

- Smartphone?
- Cloud-Service?
- Waschmaschine?
- Smart-TV?
- Smart-Home-Anwendung?

Übersicht über aktuelle Vorlesung

Hochschule für Wirtschaft und Recht Berlin
Berlin School of Economics and Law

- Organisatorisches
- Eingebettete Systeme
  - Was sind eingebettete Systeme?
  - Warum sind eingebettete Systeme wichtig?
  - Besonderheiten eingebetteter Software
  - Herausforderungen im Software Engineering
- Vorlesungsinhalte
- Literatur

Relevanz eingebetteter Systeme

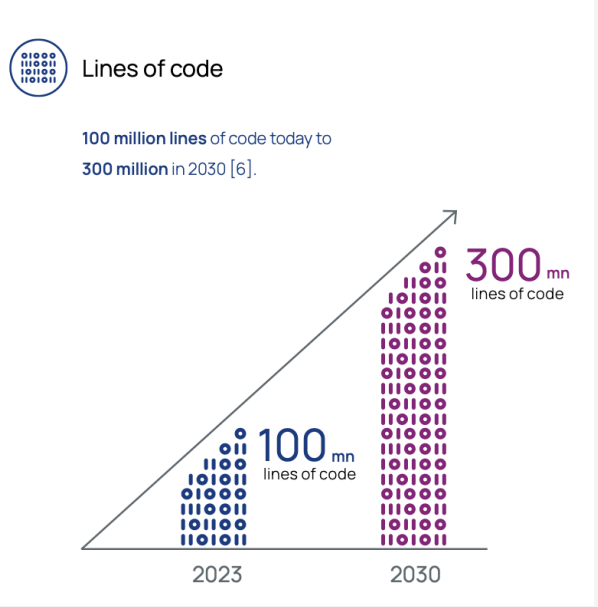
mehr als 98% aller Prozessoren weltweit in eingebetteten Systemen
100x mehr eingebetteter Programmcode als andere Software

Beispiel: PKW gehobener Ausstattung:

Jahr 2003: 70 MB eingebettete Software
Jahr 2010: 1 GB eingebettete Software, 10 Mio. LOCs
Jahr 2015: 10 GB eingebettete Software, 100 Mio. LOCs

Elektronikanteil der Automobilkosten:

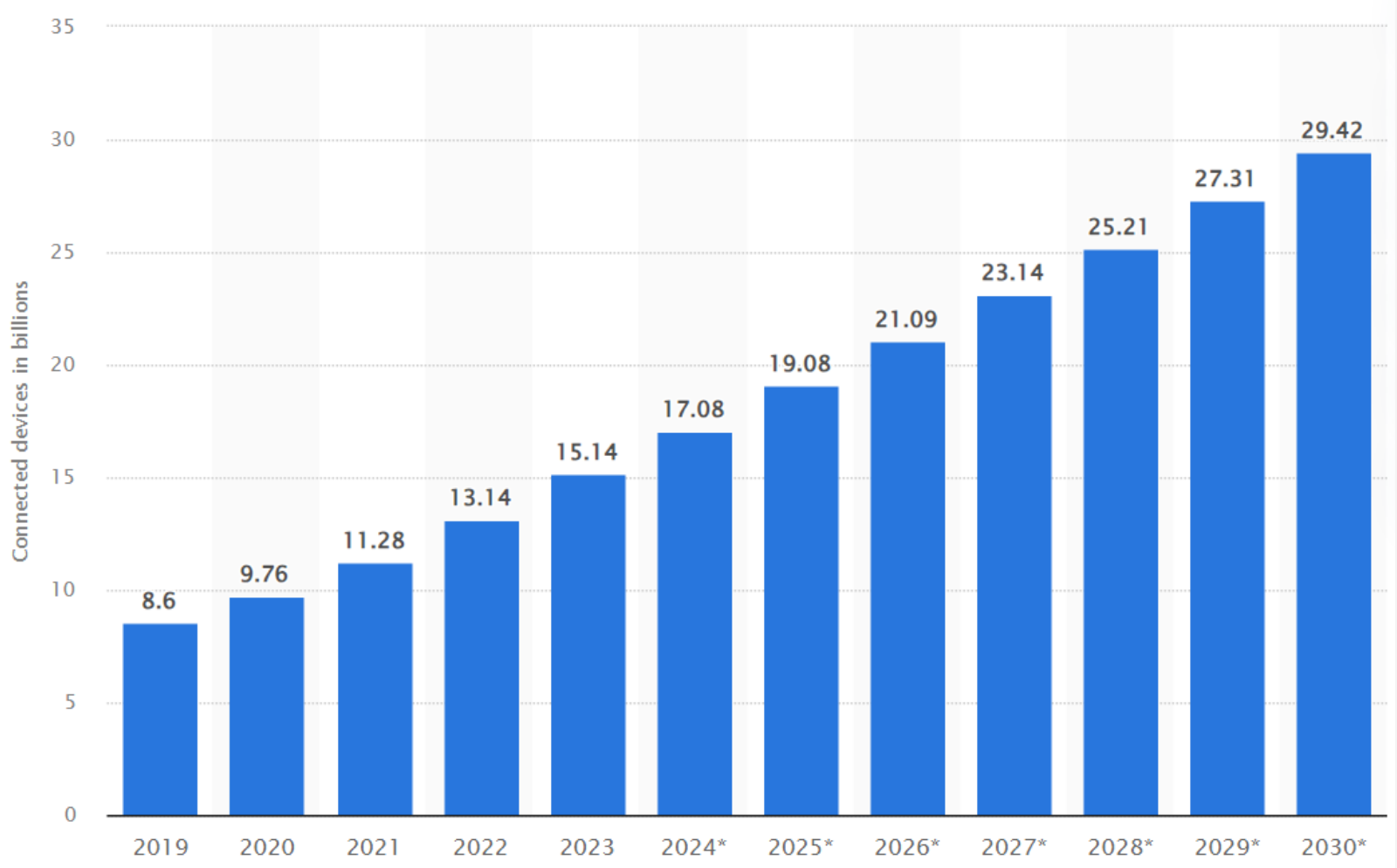
22% bis 25% im Jahr 2000
40% im Jahr 2015



Quelle: https://www.etas.com/ww/media/a_downloads/etas-whitepaper-trends-towards-software-defined-vehicles.pdf

IoT: Number of connected devices worldwide

Hochschule für Wirtschaft und Recht Berlin
Berlin School of Economics and Law



in Milliarden
* Vorhersage
für 2024 - 2030

© Statista 2023

Anwendungen

Übersicht über aktuelle Vorlesung

Hochschule für Wirtschaft und Recht Berlin
Berlin School of Economics and Law

Organisatorisches

Eingebettete Systeme

- Was sind eingebettete Systeme?
- Warum sind eingebettete Systeme wichtig?
- Besonderheiten eingebetteter Software
- Herausforderungen im Software Engineering

Vorlesungsinhalte

Literatur

Arten von Anforderungen

Funktionale Anforderungen

- Funktion liefert mathematisch korrektes Ergebnis
- WAS soll die Funktion tun?

Nicht-funktionale Anforderungen

- Funktion genügt Randbedingungen (Zeit-/Speicherverbrauch etc)
- WIE soll die Funktion etwas tun?

Nennen Sie Beispiele für nicht-funktionale Anforderungen!

Nicht-funktionale Anforderungen

Echtzeitfähigkeit

- Einhaltung von Zeitschranken (*deadlines*) / garantierte Antwortzeiten
- harte und weiche Echtzeitanforderungen

minimaler Ressourcenverbrauch

- Energieverbrauch
- Größe
- Gewicht

Zuverlässigkeit (im Sinne von dependability)

- Ausfallsicherheit (Zuverlässigkeit im Sinne von reliability)
- Wartbarkeit (insbesondere schnelle Reparatur im Fehlerfall)
- Verfügbarkeit
- Sicherheit (safety und security)

Eingebettete Software

Klassische Sicht

Programme (Software) berechnen mathematische Funktionen

Turingmaschinen, rekursive Funktionen, …

nicht-funktionale Aspekte vernachlässigbar

Eingebettete Software

Ausführung in technischer Umgebung

Hauptaufgabe: Interaktion mit der Umgebung

nicht-funktionale Anforderungen treten in den Vordergrund

(z.B. Reaktivität und Echtzeitfähigkeit)

Eingebettete Software ist anders

Aktoren und Sensoren zur Interaktion mit der physikalischen Welt

- ⇨ Reaktivität
- ⇨ Nebenläufigkeit
- ⇨ Echtzeit

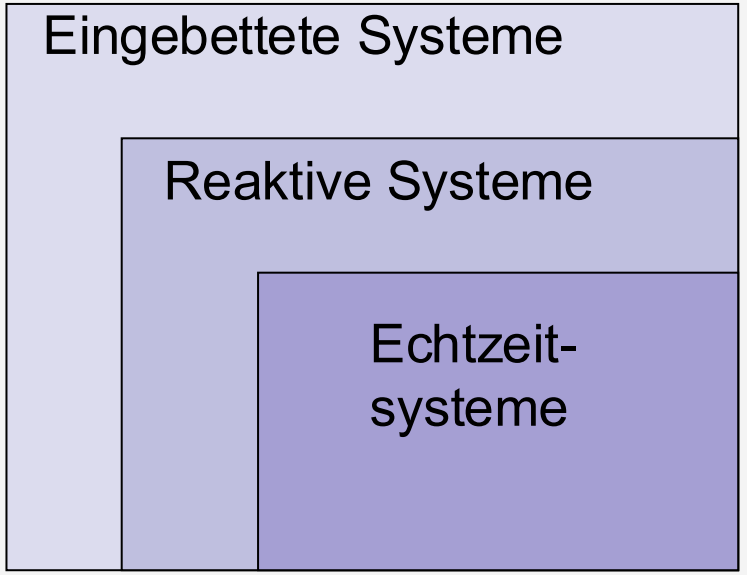
unterliegt strikten Ressourcenbeschränkungen

terminiert nicht (außer bei Fehlverhalten)

Klassifikation eingebetteter Software

- transformatorisch vs. reaktiv

- diskretes vs. kontinuierliches Zeitverhalten
- verteilte vs. monolithische Systeme
- sicherheitskritische Systeme vs. z.B. Infotainment-Systeme
- Massenmarkt-Gesetzen unterliegende Systeme, z.B. Spielekonsolen, Automobilsteuerungen



Zwischenfazit:

Eingebettete Software muss anderen Randbedingungen genügen als klassische Software.

Software Engineering eingebetteter Systeme muss diesen Randbedingungen Rechnung tragen.

Eigenschaften eingebetteter Software

- Zeitbehaftung
- Nebenläufigkeit
- Lebendigkeit
- Heterogenität
- Reaktivität
- Echtzeit

Was stellen Sie sich unter diesen Begriffen vor?
Was bedeutet das für die Softwareentwicklung?

Zeitbehaftung

Klassische Software haben keinen Zeitbegriff

Verknüpfung von Objektorientierung und Echtzeit bislang nicht zufriedenstellend
Temporallogik redet über „irgendwann“ und „immer“ (Klassische Temporallogik beschreibt nur qualitative Zeit („immer“, „irgendwann“), aber keine exakten Zeitgrenzen)

Moderne Rechnerarchitekturen

Performanzgewinne meist über statistische Speedups
(Caches, spekulative Ausführung, Sprungvorhersage)
geben dem Programmierer keine Kontrolle über das Zeitverhalten

In eingebetteten Systemen

Zeit ist immer wichtig, denn Umgebungsreaktion geschieht in Echtzeit

Nebenläufigkeit (Concurrency)

= mehr als ein Prozess (unabhängige Kontrollflüsse)

Prozesse können parallel oder beliebig verzahnt ausgeführt werden

in Eingebetteten Systemen

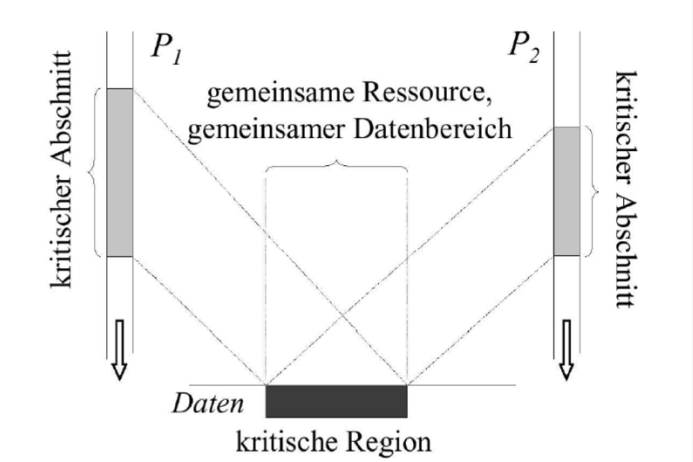
Reaktion auf Menge von Sensoren
"gleichzeitig" Steuerung der Aktoren

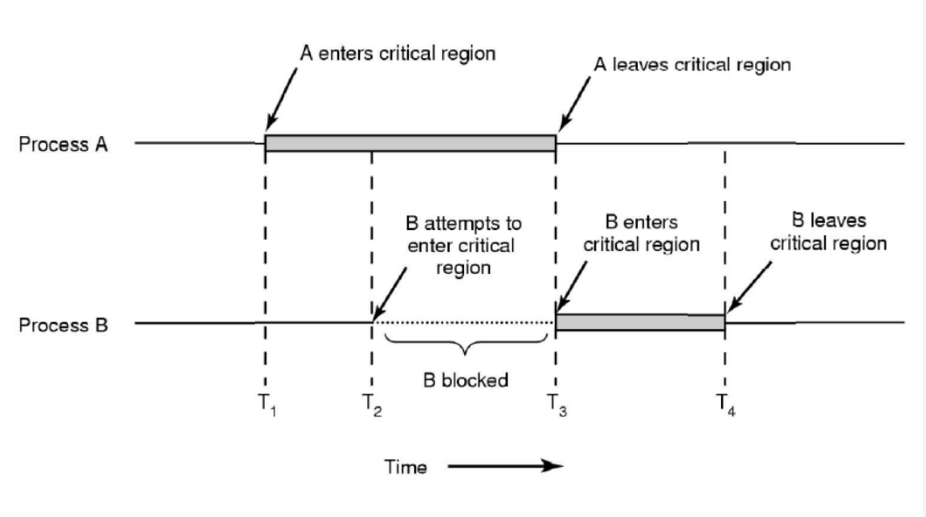
etablierte Lösungen aus der Betriebssystemwelt

Scheduling, Semaphoren, Monitore
Interrupts zur Einbeziehung der physikalischen Umgebung (externe Ereignisse)
➢schwierig, Verhalten komplexer Konstrukte vorherzusagen

Recapture: Kritische Abschnitte in Betriebssystemen

- Ein Prozess befindet sich in seinem kritischen Abschnitt, wenn er auf gemeinsame Ressourcen zugreift
- Zu jedem Zeitpunkt darf sich höchstens ein Prozess in seinem kritischen Abschnitt befinden
- Eintritt in einen kritischen Abschnitt muss geregelt erfolgen
- Kein Prozess darf in seinen kritischen Abschnitt eintreten, wenn sich bereits ein anderer Prozess in seinem kritischen Abschnitt befindet (d.h. auf Daten in einer kritischen Region zugreift)





Quelle: https://user.informatik.uni-goettingen.de/~brosenne/vortraege/os2013ws/os20131217.pdf

Recapture: Deadlock

- Im ersten Versuch: Kontrolle über flags (flag[n]==true wenn Prozess n im kritischen Abschnitt)
- Problem: Deadlock, d.h. Einfrieren

Prozess P0:

```
while (true) {
    flag[0] = true;
    while (flag[1]) /* waiting */;
    <kritischer Abschnitt>
    flag[0] = false;
    /* Rest */
  };
```

Prozess P1:

```
while (true) {
    flag[1] = true;
    while (flag[0]) /* waiting */;
    <kritischer Abschnitt>
    flag[1] = false;
    /* Rest */
  };
```

Annahme: `flag[2] = {false, false}`

P0: führt `flag[0] = true;` aus.

P1: führt `flag[1] = true;` aus.

P0: führt `while (flag[1]);` aus, da `flag[1] == true` **wartet P0**.

P1: führt `while (flag[0]);` aus, da `flag[0] == true` **wartet P1**.

****** Es kommt zu einer Verklemmung: Beide Prozesse warten darauf, dass jeweils der andere den kritischen Abschnitt wieder verlässt. ******

Quelle: https://user.informatik.uni-goettingen.de/~brosenne/vortraege/os2013ws/os20131217.pdf

Recapture: Lifelock

- Verbesserungsidee:
  - In Schleifenkörper flag zurücksetzen, um anderem Prozess den Eintritt zu ermöglichen falls der eigene Prozess in dem Moment unterbrochen wird
- Lifelock möglich (Prozesse werden weiter ausgeführt, aber mit nicht sinnvollen Fortsetzungen)

Prozess P0:

```
while (true) {
    flag[0] = true;
    while (flag[1]){
      flag[0] = false;
      /* Verzögerung */;
      flag[0] = true;
    }
    <kritischer Abschnitt>
    flag[0] = false;
    /* Rest */
  };
```

Schleifen haben diesmal Inhalt, um anderem Prozess Vorrang zu geben.

Prozess P1:

```
while (true) {
    flag[1] = true;
    while (flag[0]){
      flag[1] = false;
      /* Verzögerung */;
      flag[1] = true;
    }
    <kritischer Abschnitt>
    flag[1] = false;
    /* Rest */
  };
```

Annahme: `flag[2] = {false, false}`

P0: führt `flag[0] = true;` aus.

P1: führt `flag[1] = true;` aus.

P0: prüft `flag[1]` in `while (flag[1])`, führt Schleife aus, da `flag[1] == true`.

P1: prüft `flag[0]` in `while (flag[0])`, führt Schleife aus, da `flag[0] == true`.

P0: führt `flag[0] = false;` aus.

P1: führt `flag[1] = false;` aus.

P0: führt `flag[0] = true;` aus.

P1: führt `flag[1] = true;` aus.

****** Es kommt zu einem Livelock (dynamische Verklemmung): Erfolgreiche Ausführungssequenzen (eine Endlosschleife), aber kein Prozess kann jemals in seinen kritischen Abschnitt eintreten. ******

Quelle: https://user.informatik.uni-goettingen.de/~brosenne/vortraege/os2013ws/os20131217.pdf

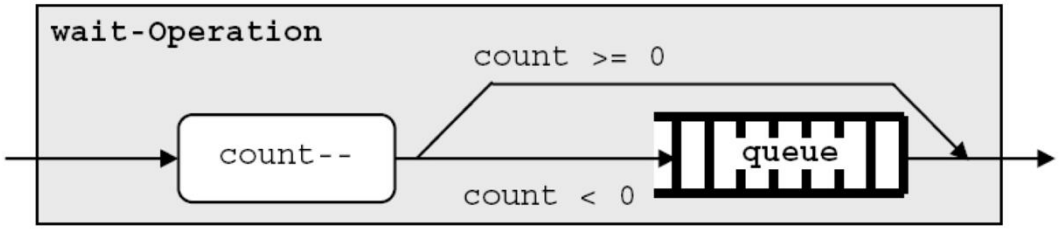
Recapture: Semaphore

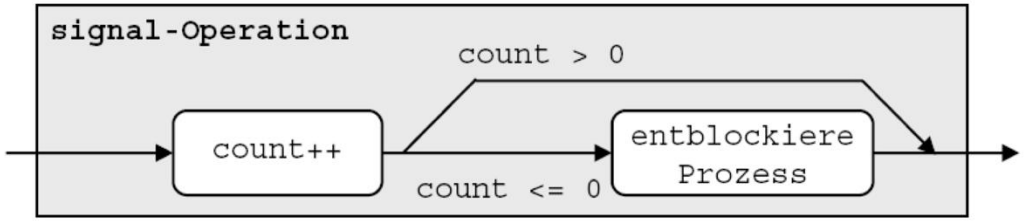
- Semaphore (Dijkstra, 1965) atomar in Hardware gelöst
- Wenn richtig angewendet, lösen sie Deadlock-Problem
- Gibt auch Software-Lösungen (Algorithmen), wie Dekker (1965) und Peterson (1981), aber schwer verifizierbar und skalierbar

```
<semaphore>.wait () {
  <semaphore>.count = <semaphore>.count - 1;
  if (<semaphore>.count < 0) {
    „diesen Prozess in <semaphore>.queue ablegen"
    „diesen Prozess blockieren"
  }
}
```

Atomare Operation

```
<semaphore>.signal () {
  <semaphore>.count = <semaphore>.count + 1;
  if (<semaphore>.count <= 0) {
    „Prozess P aus <semaphore>.queue entfernen"
    „Prozess P in Liste bereiter Prozesse ablegen"
  }
}
```





Quelle: https://user.informatik.uni-goettingen.de/~brosenne/vortraege/os2013ws/os20131217.pdf

Semaphor-Anwendung

```
const int n = /* Anzahl der Prozesse */ ;
semaphore S_WA = S_WA.init(1);   // S_WA: Sem. Wechsels. Ausschluss

void P(int i) {
  while (true) {
    S_WA.wait();        /* Eintreten oder Warten */
    /* kritischer Abschnitt */
    S_WA.signal();      /* Freigeben */
    /* Restliche Programmzeilen */
  }
}

void main() {
  parbegin (P(1), P(2), ..., P(n));
}
```

Initialisierungswert von 1 sorgt dafür, dass 1 Prozess in den kritischen Bereich kann.

Quelle: https://user.informatik.uni-goettingen.de/~brosenne/vortraege/os2013ws/os20131217.pdf

Lebendigkeit

Systeme dürfen nicht terminieren oder blockieren

⇨ unter anderem: keine Verklemmungen (deadlocks)

Deadlock

Eine Menge von Prozessen befindet sich in einem Deadlock, wenn jeder dieser Prozesse auf ein Ereignis wartet, das nur ein anderer Prozess aus dieser Menge verursachen kann.

Lebendigkeit ist essentiell in eingebetteten Systemen

Im klassischen Berechenbarkeitsmodell schlagen nicht-terminierende Programme fehl

Heterogenität

Mischung verschiedener Berechnungsmodelle (Models of Computation)

- Hardware- und Software-Anteile
- unterschiedliche Hardware, Protokolle, Ausführungsumgebungen
- Standard-Hardware und dedizierte Hardware
- kontinuierliche und diskrete Anteile
- synchrone und asynchrone Abläufe
- blockierende und nichtblockierende Kommunikation
- kontrollflussorientierte vs. datenflussorientierte Sprachen
- verschiedene Methoden, mit Ereignissen umzugehen
- ...
- ➢Spezifikationsmethoden und Programmiersprachen mit heterogener Semantik

Reaktivität

kontinuierliche Interaktion mit der Umgebung ⇨ Reaktion auf Ereignisse

„Verpassen“ von Ereignissen kann schlimme Folgen haben

Modellierung

- ereignisorientierte Modelle (Automaten und Statecharts)
- datenflussflussorientierte Sprachen (Hardwarebeschreibungssprachen, Simulink)

Programmierung

- spezielle Programmiersprachen (Ada, Esterel, Lustre)
  - verwendet in sicherheitskritischen Bereichen (Flugzeugen, Kernkraftwerken)
  - nutzen vorwiegend statische Analyseergebnisse
- Erweiterungen für Reaktivität, Echtzeit und Nebenläufigkeit in etablierten Sprachen (z.B. SystemC, POSIX für C, RTSJ für Java)

Echtzeit

Eingebettete Systeme unterliegen Echtzeitanforderungen

resultieren meist aus Interaktion mit Umgebung

häufig nicht „so schnell wie möglich“ sondern „zum richtigen Zeitpunkt“

Modellierung

zeiterweiterte Automaten (Timed Automata), kontinuierliche Blöcke in Simulink

Programmierung

clocks in Hardware, Zeitmessung und explizite *delays* in Software

Echtzeitbetriebssysteme

dynamischere Methoden, z.B. für Interrupt-Behandlung, Scheduling

Priorisierung von Prozessen

Techniken zur Einhaltung von Zeitschranken (deadlines)

Herausforderungen

- Abstraktionsmechanismen wie
  - Objektorientierung
  - (terminierende) Prozeduren mit Ein- und Ausgabeparametern
- reichen nicht aus
- Stattdessen:
  - ereignisorientierte Modelle
  - Modelle für Prozesse statt für Prozeduren
  - Einbeziehung nicht-funktionaler Eigenschaften, z.B. explizite Modellierung von Zeit

Kurze Wiederholung: Formale Semantiken

Warum meint ihr braucht man das?
Formale Semantiken, formale Verifikation?

Kurze Wiederholung: Formale Semantiken

- Sichere Systeme zu entwickeln ist faktisch unmöglich
- Systeme müssen verifiziert und validiert werden (Unterschied?)
- Verifikation: Testen und formales Beweisen
- Testen trägt oft die Hauptlast, ist aber unvollständig
- Der Zustandsraum ist oft unendlich groß → formales Beweisen in sehr sicherheitskritischen Fällen notwendig (aber sehr aufwendig)
- Um formal beweisen zu können, braucht man formale Semantiken, also klare Regeln, wie Programme ausgeführt werden
- In Embedded Systems besondere Bedeutung durch nichtfunktionale Anforderungen (Synchronisierung, Zugriff auf gemeinsame Bereiche, strict timing, Robustheit, Sicherheitskritikalität etc.)
- Das ist auch im Zeitalter von KI so (theoretische Schwelle kann nicht überwunden werden)

Kurze Wiederholung: formale Semantiken

Syntax vs. Semantik

- Syntax: Zeichen (z.B. Prädikatenlogik – $\mathrm{t}, \mathrm{p}(\mathrm{t}), \neg A, A \vee B, A \wedge B, A \rightarrow B, \forall x A(x), \exists x A(x)$
- Semantik: Interpretation (z.B. Prädikatenlogik – Struktur $\mathcal{M}$ mit
  - Trägermenge D, für n-stellige Funktionssymbole Funktion $f: D^n \rightarrow D$, Prädikatensymbole $p \subseteq D^n$
  - Interpretation mit Belegung $\sigma: Var \rightarrow D, \mathcal{M}, \sigma \vDash p(x)$ gdw. ….
- Kalküle
  - Versuch, um Grunde syntaktisch Formeln / Aussagen zu beweisen
  - Korrektheit: $T \vdash \varphi \Rightarrow T \vDash \varphi$
  - Vollständigkeit: $T \vDash \varphi \Rightarrow T \vdash \varphi$
  - Es ist klar, dass jedes konsistente System einer gewissen Mächtigkeit nicht vollständig ist
  - Außerdem ist schon die Prädikatenlogik erster Stufe nur semi-entscheidbar (s. auch Halteproblem) → Verifikation von Programmen erst recht semi-entscheidbar

Was heißt das nochmal alles?

Kurze Wiederholung: formale Semantiken

Semantiken von Standard-Sprachen wie WHILE

- Syntax
  - $Stmt ::= x := t$ (t Term, Zuweisung) |
  - $S1; S2$ (Hintereinanderausführung, Sequenz)
  - $if\ b\ then\ S2\ else\ S2$
  - $while\ b\ do\ S$
  - $skip$
- Semantik: States – Σ: Var → Int z.B.

Operationale (Small Step) Semantik

$$\text{Ass}_{\text{SS}}: \langle x \texttt{ := } a,\ \sigma\rangle \rightarrow_1 \langle \texttt{skip},\ \sigma[x \mapsto \mathcal{A}\,[\![a]\!]\,\sigma]\rangle$$

$$\text{Seq1}_{\text{SS}}: \frac{\langle c_0,\ \sigma\rangle \rightarrow_1 \langle c_0',\ \sigma'\rangle}{\langle c_0\texttt{;}\ c_1,\ \sigma\rangle \rightarrow_1 \langle c_0'\texttt{;}\ c_1,\ \sigma'\rangle} \qquad \text{Seq2}_{\text{SS}}: \langle \texttt{skip;}\ c,\ \sigma\rangle \rightarrow_1 \langle c,\ \sigma\rangle$$

$$\text{IfTT}_{\text{SS}}: \frac{\mathcal{B}\,[\![b]\!]\,\sigma = \mathbf{tt}}{\langle \texttt{if (}b\texttt{) then } c_0 \texttt{ else } c_1,\ \sigma\rangle \rightarrow_1 \langle c_0,\ \sigma\rangle} \qquad \text{IfFF}_{\text{SS}}: \frac{\mathcal{B}\,[\![b]\!]\,\sigma = \mathbf{ff}}{\langle \texttt{if (}b\texttt{) then } c_0 \texttt{ else } c_1,\ \sigma\rangle \rightarrow_1 \langle c_1,\ \sigma\rangle}$$

$$\text{While}_{\text{SS}}: \langle \texttt{while (}b\texttt{) do } c,\ \sigma\rangle \rightarrow_1 \langle \texttt{if (}b\texttt{) then } c\texttt{; while (}b\texttt{) do } c \texttt{ else skip},\ \sigma\rangle$$

- Semantik wird im Kalkülstil definiert und erklärt präzise die Ausführung des Programms
- Es gibt auch andere Semantiken (wie die denotationelle, die die Ausführung als Funktion einer partiellen Abbildung von Zustandsraumänderungen auffasst, also $[\![.]\!]: \Sigma \rightarrow \Sigma$)
- Bei Embedded Systems gibt es domänenspezifische Sprachen; da verschiebt sich der Fokus auf andere Aspekte, die Anforderungen von ES näher kommen

Quelle: https://pp.ipd.kit.edu/lehre/SS2010/semantik/skript/skript.pdf

Kurze Wiederholung: formale Semantiken

- Formale Semantiken werden in Logiken und Kalkülen verwendet, um dann Eigenschaften von Programmen zu beweisen, z.B. der Hoare-Kalkül auf klassischen, sequentiellen Programmen

$$\{ P \} \; S \; \{ Q \}$$

Vorbedingung

Nachbedingung

Beispiele für Regeln aus dem Hoare-Kalkül (Sequenz und Schleifen)

$$\frac{\{ P \} \; S_1 \; \{ Q \}; \; \{ Q \} \; S_2 \; \{ R \}}{\{ P \} \; S_1; S_2 \; \{ R \}}$$

$$\frac{\{ I \wedge B \} \; S \; \{ I \}}{\{ I \} \; \texttt{while} \; B \; \texttt{do} \; S \; \{ I \wedge \neg B \}}$$

- In ES gibt es spezielle Logiken und Kalküle, um deren nicht-funktionalen Anforderungen gerecht zu werden, z.B. Timed Logic

Quelle: https://www.uni-trier.de/fileadmin/fb4/prof/INF/TIN/Folien/ADS/ss_2008/Hoare-Kalkuel-Gellner.pdf

Herausforderungen

- Experten der Anwendungsdomäne:
  - haben genaue Kenntnis des Gesamtsystems
  - entwickeln mehr oder weniger effiziente Lösungen in kurzer Zeit
  - nutzen keine oder nur wenig Abstraktionsmethoden
- aber:
  - eingebettete Systeme werden immer komplexer

➢ Abstraktionsmethoden aus Softwaretechnik dringend notwendig

Herausforderungen

- Heterogenität
  - unterschiedliche Modelle, Sprachen und Systeme
  - dedizierte Hardware, Software, Netzwerke, Protokolle
  - physikalische Umgebung und diskrete Steuerung
- Komplexität
  - Reaktivität, Nebenläufigkeit, Echtzeit
  - zunehmende Anzahl an Komponenten und Datenmengen
  - Vernetzung, Autonomie, dynamische Umgebungen

verkürzte „Time to Market“
verlängerte Lebenszyklen der Produkte

Verbindung zu Praxis: Von Modell zur Implementierung in ROS2, Ausblick für Praxisphase

Hochschule für Wirtschaft und Recht Berlin
Berlin School of Economics and Law

1. Vom Statechart → ROS2 Controller Node

- StateChart beschreibt
  - Betriebszustände
  - Übergänge
  - Ereignisse
  - Aktionen
- Z.B.
  - Idle → Explore → AvoidObstacle → Explore → Docking
  - Events obstacleDetected, goalReached, batteryLow

3. Simulink / Regelstrecke → Control Loop

- Simulink modelliert Sollwert → Regler → System → Messung → Rückkopplung
- Beispiel: Roboter soll 0.3 m/s fahren

2. Timed Automata → Safety Monitor / Watchdog

- Timed Automata modellieren
  - Deadlines
  - Zeitlimits
  - Watchdogs
  - Timeout-Verhalten
- Safety entsteht oft durch Überwachung, nicht durch Hauptlogik (Safety Monitor Node getrennt vom Controller)
- Z.B. Wenn kein Sensorupdate < 100 ms → Stop

Nebenläufigkeit & Synchronisation - das Grundproblem moderner Systeme

Hochschule für Wirtschaft und Recht Berlin
Berlin School of Economics and Law

Eingebettete Systeme sind kein Sonderfall – sie machen ein universelles Problem explizit.

Zentrale Beobachtung

- moderne Systeme bestehen aus **mehreren gleichzeitig aktiven Einheiten**
- diese Einheiten
- reagieren auf Ereignisse
- teilen Zustand **oder**
- kommunizieren über Nachrichten
- das Verhalten ist **nicht mehr deterministisch ohne explizite Synchronisation**

Domäne	Aktive Einheit	Ereignis
Embedded Systems	Task / ISR	Interrupt
Frontend	Event Handler	User Input
Microservices	Service	Request
Datenbanken	Transaktion	Commit
Kubernetes	Pod	Health / Restart

Unterschiedliche Technologien – gleiche Strukturprobleme

Nebenläufigkeit ist kein Implementierungsdetail, sondern ein Entwurfsproblem.

Zwei Arten von Korrektheit

Synchronisation bedeutet nicht überall dasselbe.

Eingebettete Systeme

- geschlossenes System
- bekannte Tasks
- bekannte Ressourcen
- bekannte Zeitannahmen
- **Korrektheit wird bewiesen**
- Deadlock-Freiheit
- Schedulability
- WCET

→ *Deterministisch, analysierbar*

Verteilte Systeme / Cloud

- offenes System
- unbekannte Teilnehmer
- Netzwerkausfälle
- partielle Sicht
- **Korrektheit wird abgeschätzt**
- Redundanz
- Consensus
- Monitoring
- Wahrscheinlichkeiten

→ probabilistisch, robust

Embedded Systems liefern das Referenzmodell für Synchronisation unter strengen Annahmen.

Synchronisation & Nebenläufigkeit – ein domänenübergreifendes Problem

Hochschule für Wirtschaft und Recht Berlin
Berlin School of Economics and Law

Nebenläufigkeit und Synchronisation sind keine Spezialprobleme einzelner Domänen – sie sind ein universelles Strukturproblem verteilter und reaktiver Systeme.

Gemeinsame Grundprobleme (alle Domänen)

- mehrere **aktive Einheiten** (Tasks, Threads, Services, Events)
- **gemeinsamer oder verteilter Zustand**
- **unbestimmte Ausführungsreihenfolge**
- **Zeit** (explizit oder implizit)
- **Ausfälle** (Crash, Verzögerung, Nichtantwort)

Gleiche Konzepte – unterschiedliche Ausprägungen

Embedded Systems	Frontend	Microservices / DB
Task / Interrupt Service Routine (ISR)	Event / Callback	Service / Request
Shared Memory	App State	Datenbank
Queue	Event Loop	Message Broker
Mutex	Single Thread	Lock / Transaction
Deadline	UX-Responsiveness	SLO / Timeout

Zentrale Entwurfsfrage

Teilen wir Zustand (Shared Memory) oder kommunizieren wir über Nachrichten (Message Passing)?

Reflexion / Übung 1

1. Was sind ’eingebettete’ Systeme? Überlegen Sie sich Beispiele und klassifizieren Sie diese nach den in der Vorlesung vorgestellten Kriterien zur Klassifikation eingebetteter Software.
2. Wie lässt sich der Aufbau eingebetteter Systeme beschreiben?
3. Was bezeichnet der Begriff *Regelstrecke*?

Reading Assignment Teil 1

Hochschule für Wirtschaft und Recht Berlin
Berlin School of Economics and Law

Edward A. Lee, "Embedded Software",
in *Advances in Computers* (M. Zelkowitz, editor), Vol. 56, Academic Press, London, 2002.
http://ptolemy.eecs.berkeley.edu/publications/papers/02/embsoft/embsoftwre.pdf
ist ein Überblicksartikel zum Thema Embedded Software und gibt einen sehr guten Einblick in die speziellen Probleme, denen die Entwickler eingebetteter Software gegenüberstehen. Lesen Sie die Abschnitte 1 bis 3 und bereiten Sie sich auf folgende Fragen vor:

1. Inwiefern unterscheidet sich Embedded Software von ’klassischer’ Software? Welche unterschiedlichen Anforderungen gibt es? Warum ist das für uns als Software-Entwickler wichtig und wie können wir damit umgehen?
2. Diskutieren Sie die Begriffe Timeliness, Concurrency, Liveness, Interfaces, Heterogeneity, Reactivity und deren Bedeutung für die Entwickler eingebetteter Systeme.
3. Welche Schwächen (laut Autor) haben die vorherrschenden Ansötze der Softwaretechnik für Embedded Software (Objektorientierung, Hardware-Beschreibungssprachen und Echtzeitbetriebssysteme)? Teilen Sie die Meinung des Autors?

Reading Assignment Teil 2

Lesen Sie nun die verbleibenden Abschnitte des Papers und bereiten Sie sich auf folgende Fragen vor:

1. Welchen Ansatz schlägt der Autor vor? Inwiefern vermeidet er die beschriebenen Schwächen der anderen Ansätze? Was halten Sie von dem Ansatz?
2. Was bedeutet der Begriff *Model of Computation* und welchem Zweck dient er?
3. Im Artikel werden die Models of Computation der folgenden Konzepte diskutiert. Notieren Sie zu jedem der genannten Konzepte 1-2 Stichpunkte, die bezeichnend für das jeweilige Model of Computation sind.
    1. Datenflussmodelle
    2. Time Triggered Architectures (TTA)
    3. Synchronous / Reactive (SR)
    4. Discrete Event models (DE)
    5. Process Networks
    6. Rendezvous
    7. Publish and Subscribe
    8. Continuous Time models (CT)
    9. Finite State Machines
4. Was sind Vor- und Nachteile der Verwendung allgemeiner oder spezialisierter Modelle zur Modellierung eingebetteter Systeme? Welche Vor- und Nachteile hat die Kombination mehrerer Modelle?

Literatur zur Vorlesung

Wir folgen keinem einzelnen Buch.

Lehrbücher:

- Peter Marwedel: Embedded System Design. Springer 2006.
- Peter Marwedel: Embedded Systems Foundations of Cyber-Physical Systems, Second Edition. Springer 2011.
- P. Liggesmeyer & D. Rombach: Software Engineering eingebetteter Systeme. Spektrum 2005.
- Wayne Wolf: Computers as Components – Principles of Embedded Computing System Design. Academic Press 2001.

Weitere Literatur im Laufe des Semesters